

Workshop No. 2 — Design Artifacts and System Modeling

Made by:
Error 418

Members:
Bermudez Guaqueta Tomas Alejandro
Cely Infante Daniel Alfonso
Juan Sebastián Gámez Ariza
Herrera Novoa David Alejandro
Adrian Alberto Diosa Benavides

Professor:
Liliana Marcela Olarte Mesa

Software Engineering II

NATIONAL UNIVERSITY OF COLOMBIA
Bogotá, 2026

CRC Cards

Class Name: HealthEntity	Superclass/Subclass: None
Responsibilities	Collaborators
Manage clinic/IPS registration data (NIT, REPS code)	User
Track legal documents and institutional status	PlatformAdmin
Maintain health service agreement history	HealthEntity

Class Name: User	Superclass/Subclass: None (Base for Roles)
Responsibilities	Collaborators
Authenticate credentials and handle secure login	Role
Manage profile information (name, email, status)	AuditLog
Enforce session timeouts (15-minute inactivity)	HealthEntity

Class Name: Patient	Superclass/Subclass: None
Responsibilities	Collaborators
Store personal demographic data and unique ID (Cédula)	ClinicalRecord
Handle Habeas Data rights (access, correction, suppression)	InformedConsent

Class Name: ClinicalRecord	Superclass/Subclass: None
Responsibilities	Collaborators
Act as the central unified medical history container	Patient
Allow incremental additions (consultations, diagnoses)	MedicalStaff (Doctor)
Prevent deletion or tampering of historical clinical data	ConsultationEntry

Class Name: ConsultationEntry	Superclass/Subclass: None
Responsibilities	Collaborators
Record specific clinical encounter details (symptoms, vital signs)	ClinicalRecord
Link medical prescriptions and diagnostic hypotheses	MedicalStaff, Prescription

Class Name: IAModule	Superclass/Subclass: None
----------------------	---------------------------

Responsibilities	Collaborators
Analyze medical images using MONAI models within 60 seconds	ConsultationEntry
Generate diagnostic suggestions without issuing final judgments	AssistantConversation
Track AI response confidence and token usage metrics	MedicalImage

Class Name: Prescription	Superclass/Subclass: None
Responsibilities	Collaborators
Store medication details, dosages, and active ingredients	ConsultationEntry
Link the issuing doctor and the recipient patient	MedicalStaff, Patient
Validate pharmaceutical codes	

Class Name: InformedConsent	Superclass/Subclass: None
Responsibilities	Collaborators
Document patient authorization for procedures or data processing	Patient
Verify digital/physical signature status and timestamps	ClinicalRecord

Class Name: AuditLog	Superclass/Subclass: None
Responsibilities	Collaborators
Record every system transaction immutably for security auditing	User
Track IP addresses, operation types, and JSON diffs	All Classes
Register unauthorized access attempts immediately	

Mockups

Business Model Processes

Drive Link:

<https://drive.google.com/file/d/1p0nOpFm5WKasOBEIxcV1ab1ggQda4Fp/view?usp=sharing>

Process Description

This process models a complete medical consultation within the SINAPSIS platform, covering from initial authentication to the final recording of the diagnosis and treatment in the patient's unified clinical record.

The process begins when an authenticated physician logs into the platform using multi-factor authentication (MFA). Once inside, the physician searches for the patient by their unique identifier (national ID). The system then checks whether the patient already has a clinical record: if not, a new one is created on the spot; if one exists, it is opened directly.

With the clinical record available, the physician registers a pre-diagnosis for the current consultation — a mandatory step that documents the physician's own clinical reasoning before any external tool is used. The physician then uploads the relevant medical image (X-ray, MRI, etc.) and decides whether to request AI support for the reading.

If the physician chooses not to use AI, the process moves directly to registering the final diagnosis and treatment. If AI support is requested, the system first verifies that a pre-diagnosis has been recorded for this consultation. If it has not, access to the AI module is blocked and the physician is prompted to register the pre-diagnosis before continuing. If the pre-diagnosis exists, the image is forwarded to the AI module (MONAI), which analyzes it and generates a suggestion along with a mandatory ethical disclaimer clarifying that the output does not constitute a diagnosis and that the physician retains full clinical responsibility. The physician reviews the suggestion, contrasts it with their own clinical judgment, and proceeds to register the final diagnosis and treatment.

Regardless of whether AI was used, the process concludes with the system automatically saving all new records to the patient's clinical history and generating an immutable audit log entry that registers who performed each action, when, and what was modified.

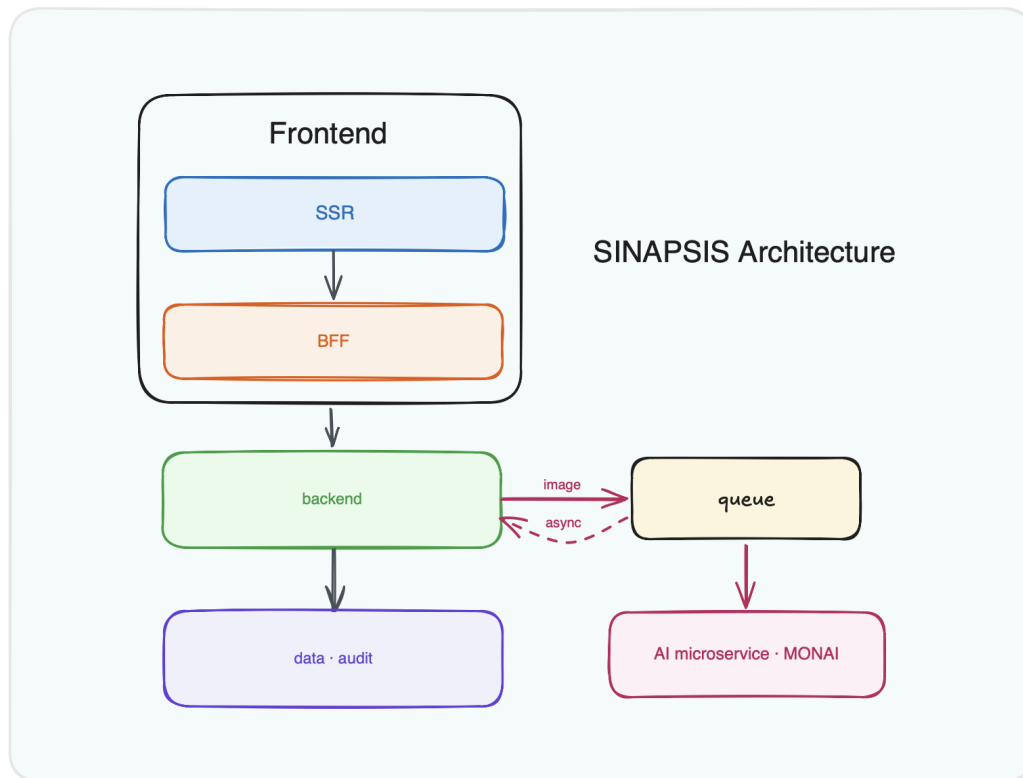
Role in the Application

This process is the core operational flow of SINAPSIS. It directly involves three of the system's four modules: the Unified Clinical Record module (Módulo 1), the AI Suggestions module (Módulo 2), and the Administration and Security module (Módulo 4) through its audit logging component.

From a business perspective, this process addresses the main problem SINAPSIS was designed to solve: fragmented and incomplete clinical information. By centralizing record creation and updates in a single authenticated flow, and by integrating AI-assisted image analysis as an optional but controlled step, the system reduces the risk of diagnostic omissions while ensuring that the physician's judgment remains the authoritative decision. The mandatory pre-diagnosis requirement before AI access enforces this principle at the process level, preventing the AI module from being used as a substitute for clinical reasoning rather than a complement to it. The automatic audit trail satisfies both internal

traceability requirements and Colombian healthcare regulations, specifically Resolución 1995 de 1999 and Ley 1581 de 2012.

Architecture Diagram



The architecture uses different technologies for each part of the system. The frontend is a React app with server-side rendering (SSR). Behind it, a small Node.js BFF handles things close to the user, like MFA, sessions, and preparing data for the screen. The main backend is written in Go, because Go can handle many requests at the same time using few resources. For data, we use a relational database for clinical records and an append-only audit log. The AI module is a separate Python microservice that uses MONAI. It is connected to the backend through a RabbitMQ queue, so the slow image analysis does not block or break the clinical system. We connect the AI microservice to the backend and not to the BFF for an important reason: the backend can apply the business rules first. Before any image goes to MONAI, the backend checks the required pre-diagnosis, checks the doctor's role, and saves the request in the audit log. These are clinical decisions, so they belong in the backend, not in the BFF. This is why the queue is connected to the backend and the BFF stays simple.



Class Diagram



Relational Database Model